

Algorithmique & Programmation

Introduction à la programmation pour le TAL

Eric Jordan

10 octobre 2025

- 1 Les strings
- 2 Opérations sur les strings
- 3 A l'action

1 Les strings

2 Opérations sur les strings

3 A l'action

Le type pour stocker du texte

str — le type des chaînes de caractères

- ▶ Appelé **string** (ou *chaîne de caractères*)
- ▶ Une **séquence** : les caractères sont ordonnés
- ▶ **Immuable** : on ne peut pas la modifier après création
- ▶ Compatible **Unicode** : supporte les accents, les emojis et les caractères de toute langue du monde

Créer une chaîne de caractères

Plusieurs façons de créer un string :

- 1 Avec des apostrophes : `'Une chaîne'`
- 2 Avec des guillemets : `"Une autre"`

Créer une chaîne de caractères

Plusieurs façons de créer un string :

- 1 Avec des apostrophes : 'Une chaîne'
- 2 Avec des guillemets : "Une autre"
- 3 Avec des triples quotes :

```
ceci = '''Comme ceci'''  
cela = """Ou comme cela"""
```

Créer une chaîne de caractères

Plusieurs façons de créer un string :

- 1 Avec des apostrophes : 'Une chaîne'
- 2 Avec des guillemets : "Une autre"
- 3 Avec des triples quotes :

```
ceci = '''Comme ceci'''  
cela = """Ou comme cela"""
```

Utile pour les textes multi-lignes ou la documentation (docstrings).

Pourquoi utiliser le triple ?

Pour écrire des chaînes de plusieurs lignes :

```
texte = """Vous savez, moi je ne crois pas'  
qu'il y ait de bonne ou de mauvaise situation.  
Moi, si je devais résumer ma vie 'aujourd'hui...  
"""
```


Pourquoi utiliser le triple ?

Pour écrire des chaînes de plusieurs lignes :

```
texte = """Vous savez, moi je ne crois pas'  
qu'il y ait de bonne ou de mauvaise situation.  
Moi, si je devais résumer ma vie 'aujourd'hui...  
"""
```

A noter : Python conserve les sauts de ligne et les espaces.

Convertir d'autres données en string

Fonction `str()`

Permet de transformer (presque) n'importe quel objet en chaîne de caractères.

```
nombre = 42
texte = str(nombre)
print(texte)           # "42"

reel = 4.2
print(str(reel))       # "4.2"

booleen = True
print(str(booleen))    # "True"
```

La chaîne vide

Il est possible de créer une chaîne sans aucun caractère : **la chaîne vide**.

```
chaine_vide = ""
```

La chaîne vide

Il est possible de créer une chaîne sans aucun caractère : **la chaîne vide**.

```
chaine_vide = ""
```

Pourquoi faire ?

- ▶ Pour construire une chaîne progressivement

```
bouteilles = 99
base = ''
base += "stock actuel : "
base += str(bouteilles)
print(base) # stock actuel : 99
```

- ▶ Pour tester si une chaîne est vide :

```
if chaine == "":
    print("Vide !")
```

Échapper des caractères spéciaux

Certains caractères ont une signification spéciale dans une chaîne. Si on en a besoin, on les **échappe** avec une barre oblique inverse :

\

Échapper des caractères spéciaux

Certains caractères ont une signification spéciale dans une chaîne. Si on en a besoin, on les **échappe** avec une barre oblique inverse :

\

```
texte = 'C\'est un bel après-midi'
print(texte)  # C'est un bel après-midi

chemin = "C:\\Users\\Alice\\Documents"
print(chemin) # C:\Users\Alice\Documents

print("Ligne 1\nLigne 2")  # \n crée un saut de ligne
print("Tab\tSéparé")       # \t insère une tabulation
```

Sommaire

1 Les strings

2 Opérations sur les strings

3 A l'action

Concaténation (+) et duplication (*)

Concaténation :

```
mot = "bon" + "jour"
```


Concaténation (+) et duplication (*)

Concaténation :

```
mot = "bon" + "jour"
```

assembler plusieurs chaînes avec +

```
print(mot)    # bonjour
```

Concaténation (+) et duplication (*)

Concaténation :

```
mot = "bon" + "jour"
```

assembler plusieurs chaînes avec +

```
print(mot)    # bonjour
```

Duplication : répéter une chaîne avec *

```
rire = "ha" * 3
```

Concaténation (+) et duplication (*)

Concaténation :

```
mot = "bon" + "jour"
```

assembler plusieurs chaînes avec +

```
print(mot)    # bonjour
```

Duplication : répéter une chaîne avec *

```
rire = "ha" * 3
```

répéter une chaîne avec *

```
print(rire)   # hahaha
```

Opérateur [] : accès aux caractères

On peut accéder à un caractère précis d'une chaîne grâce à son **indice** :

```
mot = "Python"  
print(mot[0])  
print(mot[1])  
print(mot[5])  
print(mot[6])
```

Opérateur [] : accès aux caractères

On peut accéder à un caractère précis d'une chaîne grâce à son **indice** :

```
mot = "Python"  
print(mot[0])  
print(mot[1])  
print(mot[5])  
print(mot[6])
```

Les indices commencent à 0 :

P(0) y(1) t(2) h(3) o(4) n(5)

Opérateur [] : accès aux caractères

On peut accéder à un caractère précis d'une chaîne grâce à son **indice** :

```
mot = "Python"  
print(mot[0])  
print(mot[1])  
print(mot[5])  
print(mot[6])
```

Les indices commencent à 0 :

P(0) y(1) t(2) h(3) o(4) n(5)

On peut aussi accéder depuis la fin avec des indices négatifs :

```
print(mot[-1])    # n  
print(mot[-2])    # o
```

Opérateur [::] : slicing

Le slicing permet d'extraire une **sous-chaîne** :

```
chaîne[début:fin:pas]
```

- ▶ début : l'indice de départ (inclus)
- ▶ fin : l'indice d'arrêt (exclue)
- ▶ pas : intervalle entre caractères (optionnel, 1 par défaut) - utilisé pour «sauter» des caractères ou changer le «sens» de parcours

Opérateur [::] : slicing

Le slicing permet d'extraire une **sous-chaîne** :

chaîne[début:fin:pas]

- ▶ début : l'indice de départ (inclus)
- ▶ fin : l'indice d'arrêt (exclue)
- ▶ pas : intervalle entre caractères (optionnel, 1 par défaut) - utilisé pour «sauter» des caractères ou changer le «sens» de parcours

```
alphabet = "abcdefghijklmnopqrstuvwxyz"
```

```
print(alphabet[0:5])      # abcde
print(alphabet[:5])       # abcde (depuis le début)
print(alphabet[20:])      # uvwxyz (jusqu'à la fin)
print(alphabet[::2])      # acegikmoqsuwy
print(alphabet[::-1])     # zyxwvutsrqponmlkjihgfedcba
```


Opérateur : in

Permet de tester si une **sous-chaîne** est contenue dans une autre.

```
mot = "programmation"  
  
print("gram" in mot)      # True  
print("z" in mot)         # False
```

Opérateur : in

Permet de tester si une **sous-chaîne** est contenue dans une autre.

```
mot = "programmation"

print("gram" in mot)      # True
print("z" in mot)        # False
```

On peut aussi tester le contraire avec `not in`:

```
if "java" not in mot:
    print("Ce n'est pas du Java !")
```

Sommaire

1 Les strings

2 Opérations sur les strings

3 A l'action

Exercices

- 1 On a 7 prénoms : Jack, Kack, Lack, Mack, Nack, Oack, Pack et Qack. Ecrivez un petit script qui génère tous ces noms à partir des deux chaînes suivantes : prefixes = 'JKLMNOPQ' et suffixe = 'ack'
- 2 Ecrivez un script qui détermine si une chaîne contient ou non le caractère «e»
- 3 Ecrivez un script qui parcourt une chaîne : lorsqu'il rencontre une voyelle, il l'affiche, si c'est une consonne, il met juste «*»
- 4 Ecrivez un script qui recopie une chaîne (dans une nouvelle variable), en insérant des astérisques entre les caractères
- 5 Ecrivez un script qui recopie une chaîne (dans une nouvelle variable) en l'inversant

Wordle (Une description complète se trouve ici)

Le joueur doit deviner un mot de 5 lettres en 6 essais. À chaque essai, le jeu indique pour chaque lettre :

- ▶  si elle est correcte et bien placée
- ▶  La lettre en minuscule si elle est dans le mot, mais mal placée
- ▶  si elle n'apparaît pas dans le mot

Par exemple, avec mot cible arbre :

poule →     

carte →     

arbre →     